

Mall Customer Value Segmentation and Precision Marketing Strategy Analysis Report

— Based on K-Means Clustering Algorithm

Analysis Subject: Mall member consumption data

Prepared by: Liu Shiqi

Generated Date: September 20, 2026

```
In [1]: # ----- Data Processing -----
import pandas as pd
import numpy as np

# ----- Visualization -----
import matplotlib.pyplot as plt
import seaborn as sns

# ----- Machine Learning -----
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# ----- Optional (uncomment if installed) -----
# from yellowbrick.cluster import KElbowVisualizer, SilhouetteVisualizer

# ----- Global Style -----
sns.set_style("whitegrid")
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 10
```

```
In [2]: # Load dataset (replace with your local file path)
df = pd.read_csv(r"C:\Users\Administrator\Downloads\Mall_Customers.csv")

# Rename columns for easier coding
df.columns = ['CustomerID', 'Gender', 'Age', 'Annual_Income', 'Spending_Score']

# Basic inspection
print("=== Dataset Shape ===")
print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")

print("\n=== First 5 Rows ===")
display(df.head())

print("\n=== Data Types & Non-Null Counts ===")
df.info()

print("\n=== Numerical Statistics ===")
display(df.describe().round(2))
```

=== Dataset Shape ===
Rows: 200, Columns: 5

=== First 5 Rows ===

	CustomerID	Gender	Age	Annual_Income	Spending_Score
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

=== Data Types & Non-Null Counts ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
Column Non-Null Count Dtype
--- -----
0 CustomerID 200 non-null int64
1 Gender 200 non-null object
2 Age 200 non-null int64
3 Annual_Income 200 non-null int64
4 Spending_Score 200 non-null int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB

=== Numerical Statistics ===

	CustomerID	Age	Annual_Income	Spending_Score
count	200.00	200.00	200.00	200.00
mean	100.50	38.85	60.56	50.20
std	57.88	13.97	26.26	25.82
min	1.00	18.00	15.00	1.00
25%	50.75	28.75	41.50	34.75
50%	100.50	36.00	61.50	50.00
75%	150.25	49.00	78.00	73.00
max	200.00	70.00	137.00	99.00

```
In [3]: # Count missing values per column
print("Missing values per column:")
print(df.isnull().sum())

# Uncomment below if missing values exist
# # Fill numerical columns with median
# df['Age'].fillna(df['Age'].median(), inplace=True)
# df['Annual_Income'].fillna(df['Annual_Income'].median(), inplace=True)
# df['Spending_Score'].fillna(df['Spending_Score'].median(), inplace=True)
# # Fill categorical column with mode
# df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)
```

Missing values per column:

```
CustomerID      0
Gender          0
Age             0
Annual_Income   0
Spending_Score  0
dtype: int64
```

```
In [4]: duplicate_num = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicate_num}")

if duplicate_num > 0:
    df.drop_duplicates(inplace=True)
    print(f"Duplicates removed. Remaining rows: {df.shape[0]}")
```

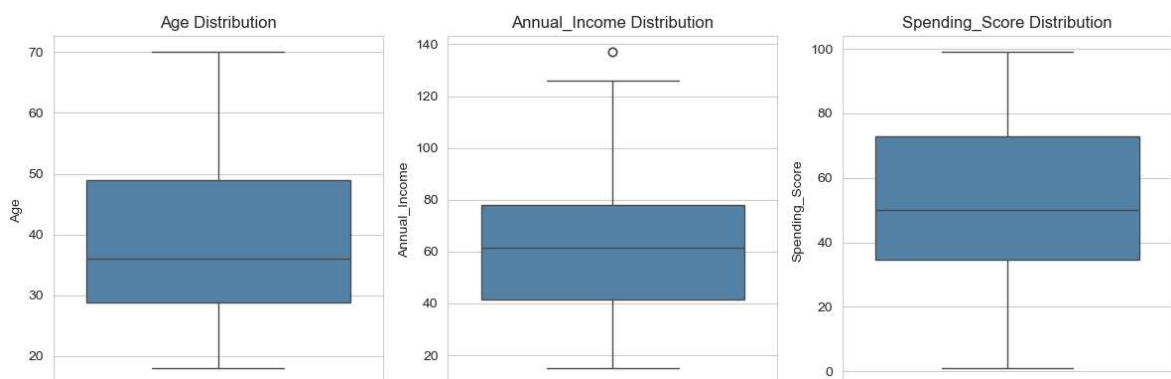
Number of duplicate rows: 0

```
In [5]: # Boxplot for numerical columns
plt.figure(figsize=(12, 4))

for i, col in enumerate(['Age', 'Annual_Income', 'Spending_Score'], 1):
    plt.subplot(1, 3, i)
    sns.boxplot(y=df[col], color='steelblue')
    plt.title(f'{col} Distribution')

plt.tight_layout()
plt.show()

# Note: Outliers are kept for business meaning (high-value customers)
```



```
In [6]: # Drop ID column (no analytical value)
df.drop('CustomerID', axis=1, inplace=True)

# Encode Gender: Male=0, Female=1
df['Gender'] = df['Gender'].map({'Male': 0, 'Female': 1})

# Verify result
print("=== Preprocessed Data Preview ===")
display(df.head())
```

=== Preprocessed Data Preview ===

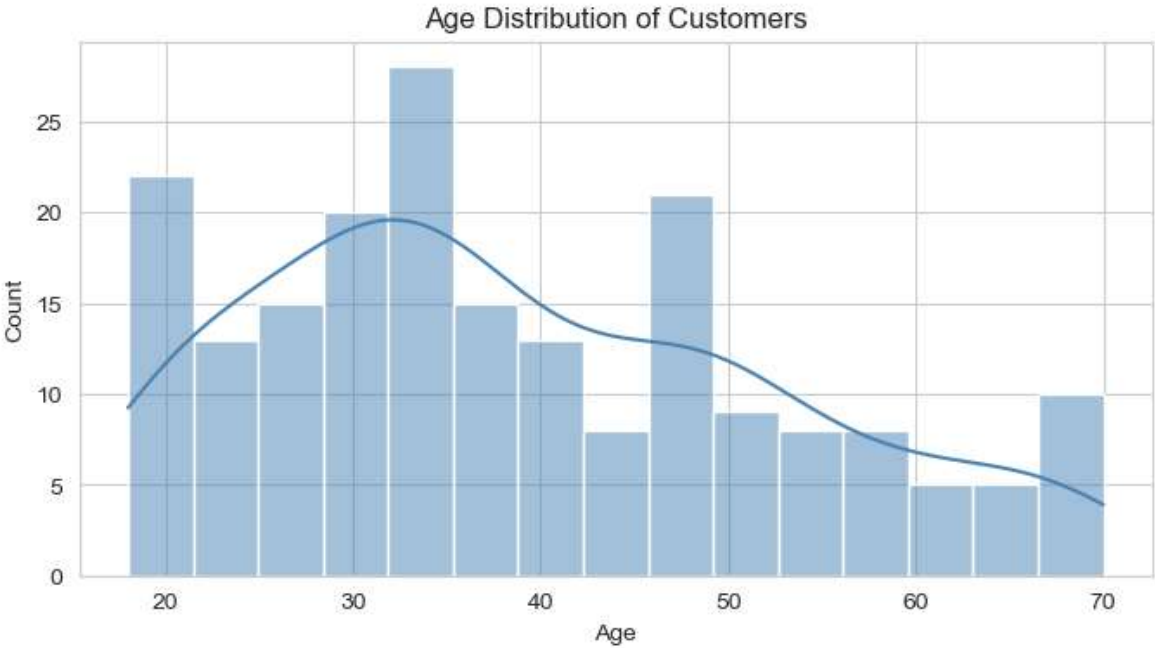
	Gender	Age	Annual_Income	Spending_Score
0	0	19	15	39
1	0	21	15	81
2	1	20	16	6
3	1	23	16	77
4	1	31	17	40

```
In [7]: # Age distribution
plt.figure(figsize=(8, 4))
sns.histplot(df['Age'], kde=True, bins=15, color='steelblue')
plt.title('Age Distribution of Customers')
plt.xlabel('Age')
plt.ylabel('Count')
plt.show()

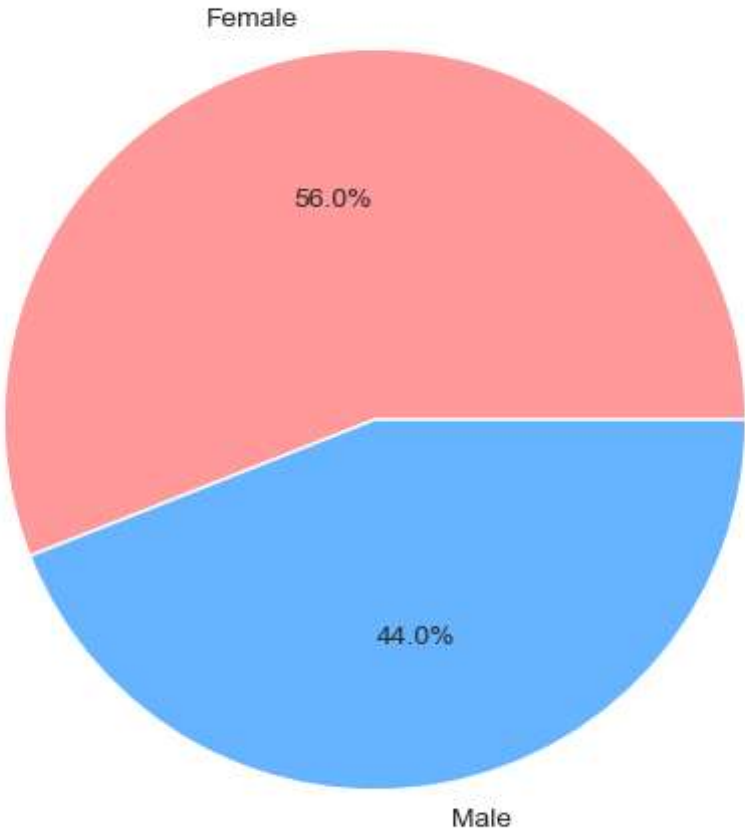
# Gender ratio
plt.figure(figsize=(6, 6))
gender_counts = df['Gender'].value_counts()
plt.pie(gender_counts, labels=['Female', 'Male'], autopct='%.1f%%', colors=['#ff
plt.title('Customer Gender Ratio')
plt.show()

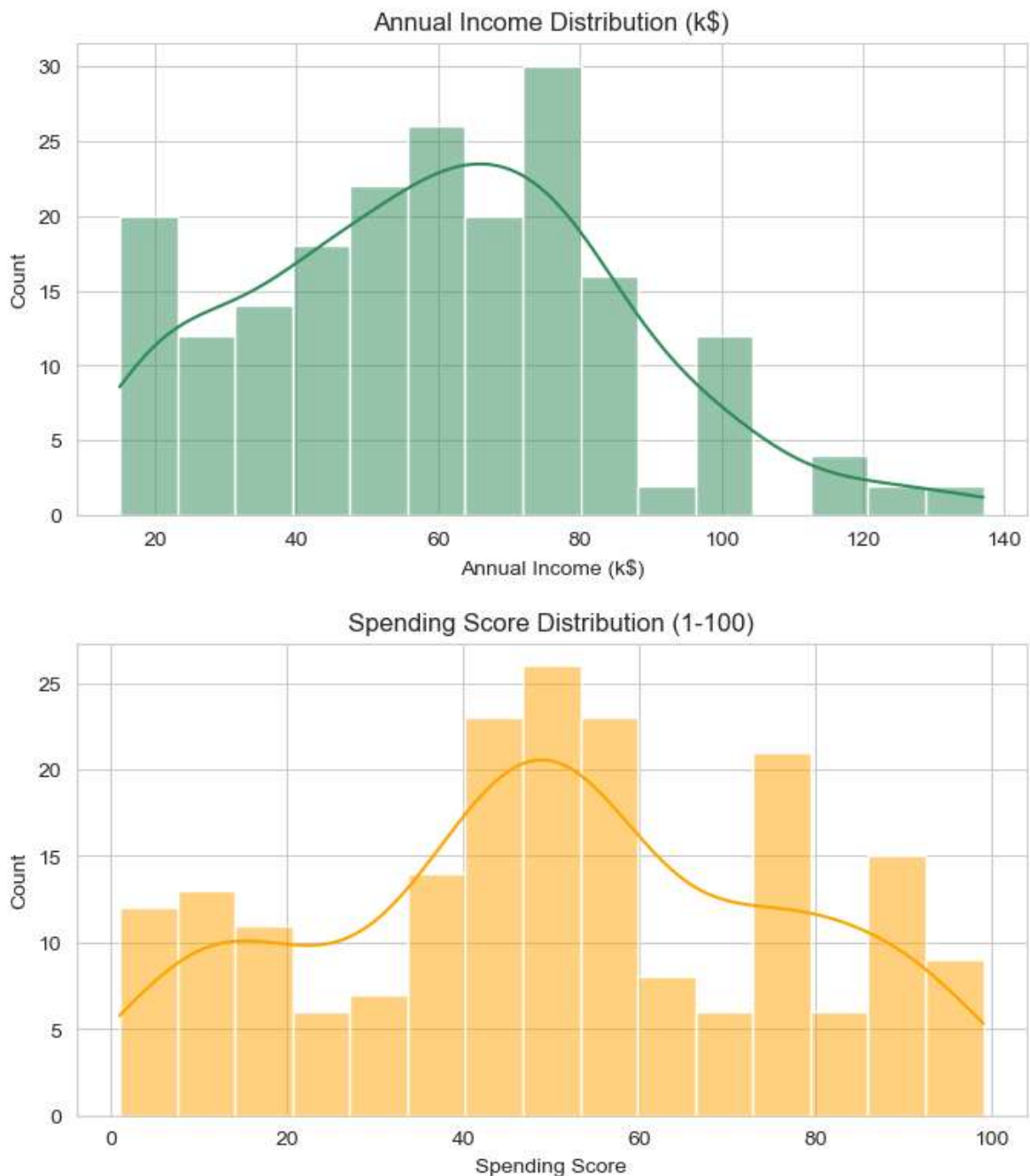
# Annual Income distribution
plt.figure(figsize=(8, 4))
sns.histplot(df['Annual_Income'], kde=True, bins=15, color='seagreen')
plt.title('Annual Income Distribution (k$)')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Count')
plt.show()

# Spending Score distribution
plt.figure(figsize=(8, 4))
sns.histplot(df['Spending_Score'], kde=True, bins=15, color='orange')
plt.title('Spending Score Distribution (1-100)')
plt.xlabel('Spending Score')
plt.ylabel('Count')
plt.show()
```



Customer Gender Ratio





```
In [8]: # Core plot: Income vs Spending Score
plt.figure(figsize=(8, 6))
sns.scatterplot(x='Annual_Income', y='Spending_Score', data=df, s=60, alpha=0.7)
plt.title('Annual Income vs Spending Score')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.show()

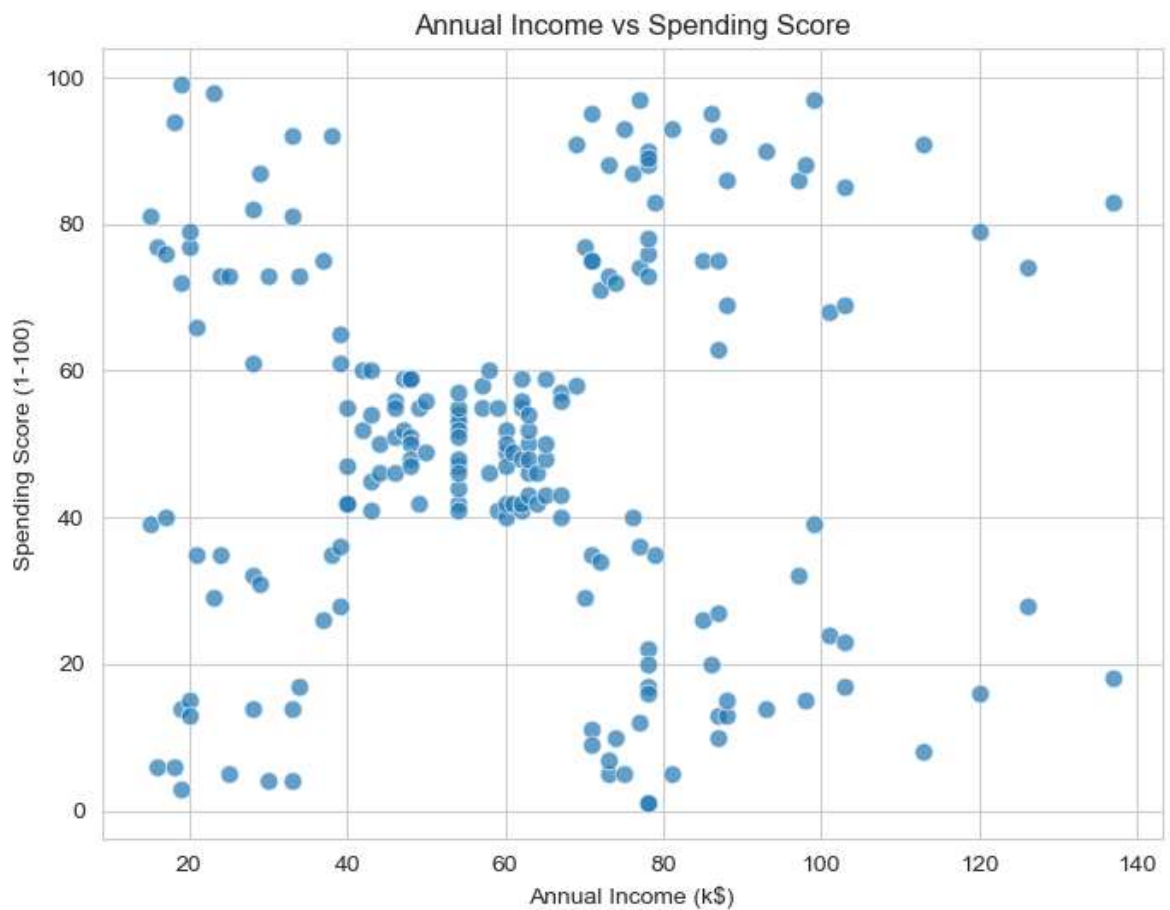
# Age vs Spending Score
plt.figure(figsize=(8, 6))
sns.scatterplot(x='Age', y='Spending_Score', data=df, s=60, alpha=0.7, color='pu')
plt.title('Age vs Spending Score')
plt.xlabel('Age')
plt.ylabel('Spending Score (1-100)')
plt.show()

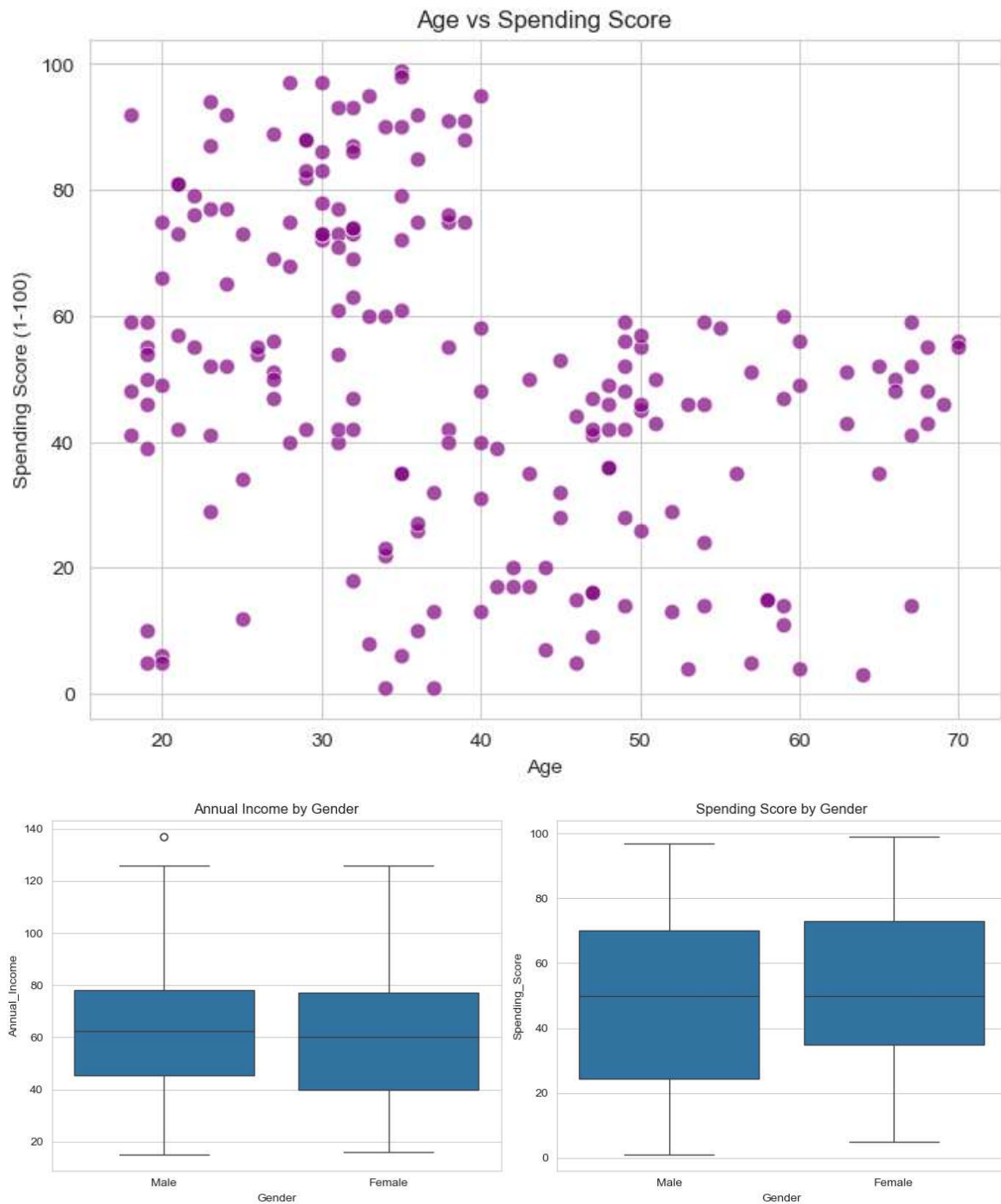
# Gender comparison
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
sns.boxplot(x='Gender', y='Annual_Income', data=df)
plt.title('Annual Income by Gender')
plt.xticks([0, 1], ['Male', 'Female'])

plt.subplot(1, 2, 2)
sns.boxplot(x='Gender', y='Spending_Score', data=df)
plt.title('Spending Score by Gender')
plt.xticks([0, 1], ['Male', 'Female'])

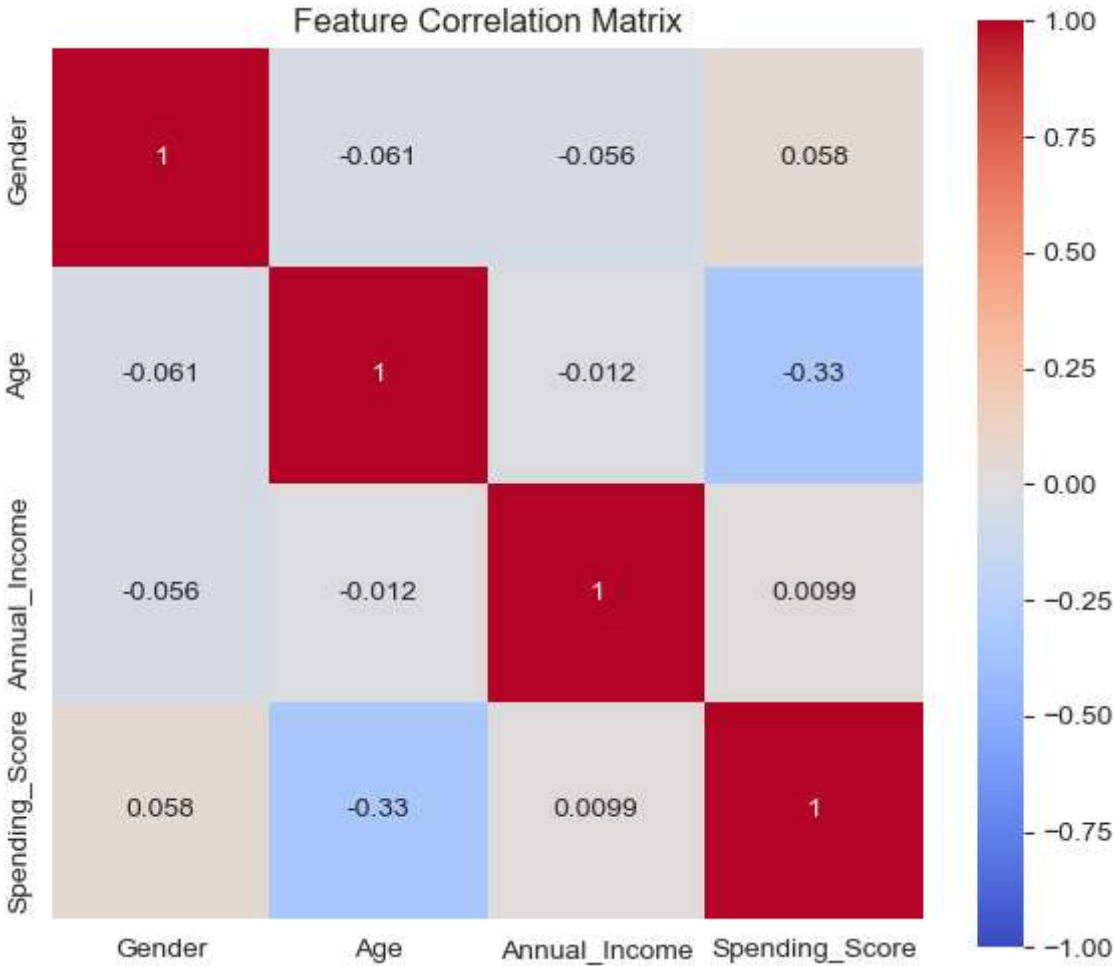
plt.tight_layout()
plt.show()
```





```
In [9]: corr_matrix = df.corr()

plt.figure(figsize=(7, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1, square=True)
plt.title('Feature Correlation Matrix')
plt.show()
```



Key EDA Observations

- 1. There is no strong linear correlation between annual income and spending score. High-income customers show significant consumption divergence.
- 2. Customers aged 25-35 generally have higher spending willingness than older age groups.
- 3. The income-spending scatter plot shows clear natural clusters, which supports the feasibility of K-Means segmentation.

```
In [10]: # Select core features for 2D clustering (most interpretable for business)
X = df[['Annual_Income', 'Spending_Score']]
print(f"Modeling data shape: {X.shape}")
```

Modeling data shape: (200, 2)

```
In [11]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Verify standardization (mean ≈ 0, std ≈ 1)
print("Mean after scaling:", X_scaled.mean(axis=0).round(4))
print("Std after scaling:", X_scaled.std(axis=0).round(4))
```

Mean after scaling: [-0. -0.]
Std after scaling: [1. 1.]

```
In [12]: sse = []
k_range = range(2, 11)
```

```

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    sse.append(kmeans.inertia_)

# Plot elbow curve
plt.figure(figsize=(8, 4))
plt.plot(k_range, sse, 'bo-', linewidth=2)
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Sum of Squared Errors (Inertia)')
plt.title('Elbow Method for Optimal k')
plt.xticks(k_range)
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

```

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use rWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.

warnings.warn(

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use rWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.

warnings.warn(

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use rWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.

warnings.warn(

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use rWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.

warnings.warn(

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use rWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.

warnings.warn(

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use rWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.

warnings.warn(

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use rWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.

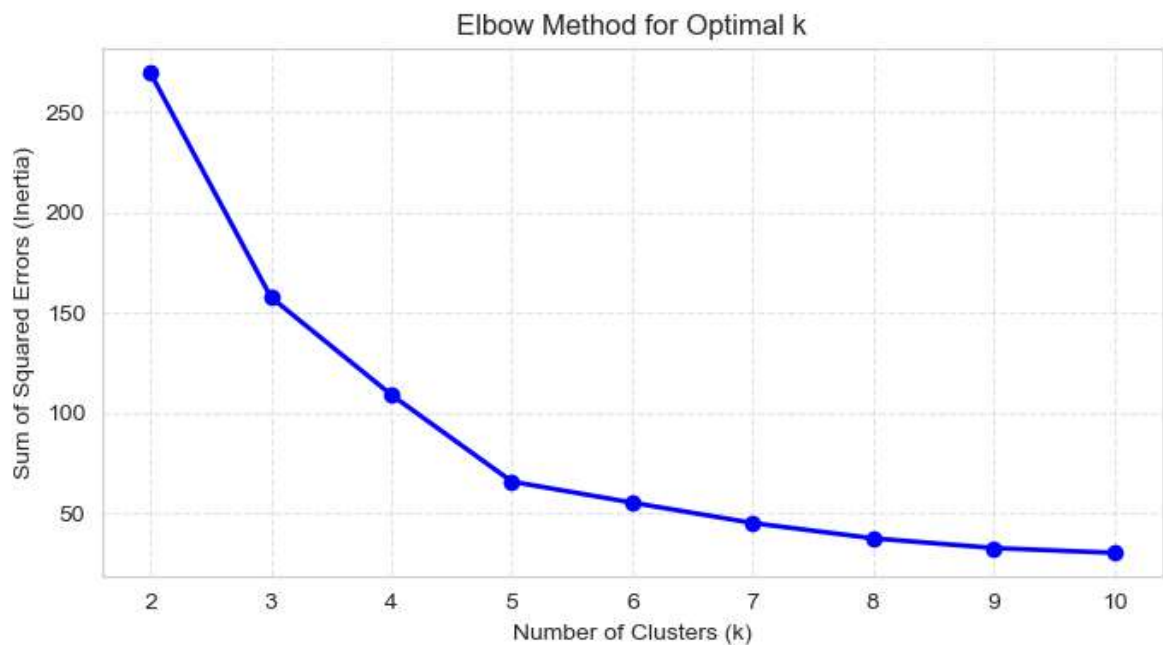
warnings.warn(

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use rWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.

warnings.warn(

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use rWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.

warnings.warn(

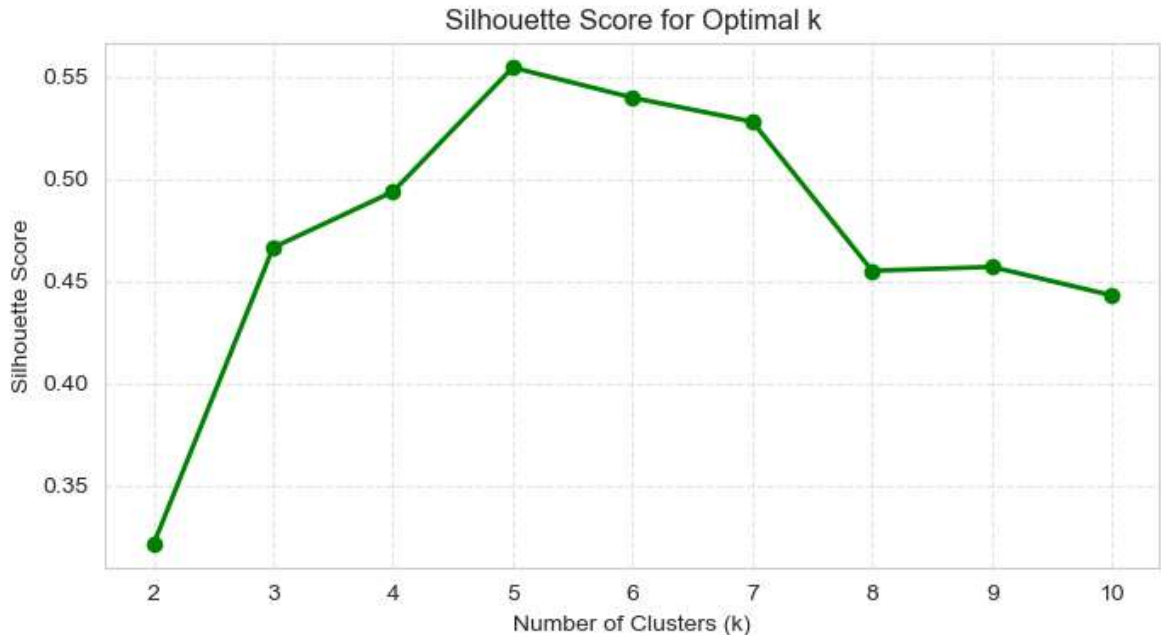


```
In [13]: sil_scores = []

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    sil_scores.append(score)
    print(f"k = {k} | Silhouette Score = {score:.4f}")

# Plot silhouette scores
plt.figure(figsize=(8, 4))
plt.plot(k_range, sil_scores, 'go-', linewidth=2)
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Score for Optimal k')
plt.xticks(k_range)
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

```
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
  warnings.warn(
k = 2 | Silhouette Score = 0.3213
k = 3 | Silhouette Score = 0.4666
k = 4 | Silhouette Score = 0.4939
k = 5 | Silhouette Score = 0.5547
k = 6 | Silhouette Score = 0.5399
k = 7 | Silhouette Score = 0.5281
k = 8 | Silhouette Score = 0.4552
k = 9 | Silhouette Score = 0.4571
k = 10 | Silhouette Score = 0.4432
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
  warnings.warn(
```



```
In [14]: # Set optimal k (typically 5 for this dataset)
         optimal_k = 5

         # Train final model
         kmeans_final = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
         kmeans_final.fit(X_scaled)

         # Assign cluster labels to original data
         df['Cluster'] = kmeans_final.labels_

         # Show customer count per cluster
         print("Customer count per cluster:")
         print(df['Cluster'].value_counts().sort_index())
```

Customer count per cluster:

```
Cluster
0      81
1      39
2      22
3      35
4      23
```

Name: count, dtype: int64

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
warnings.warn(

```
In [15]: plt.figure(figsize=(9, 7))

         # Plot data points colored by cluster
         sns.scatterplot(
             x='Annual_Income', y='Spending_Score',
             hue='Cluster', data=df,
             palette='Set1', s=80, alpha=0.8, legend='full'
         )

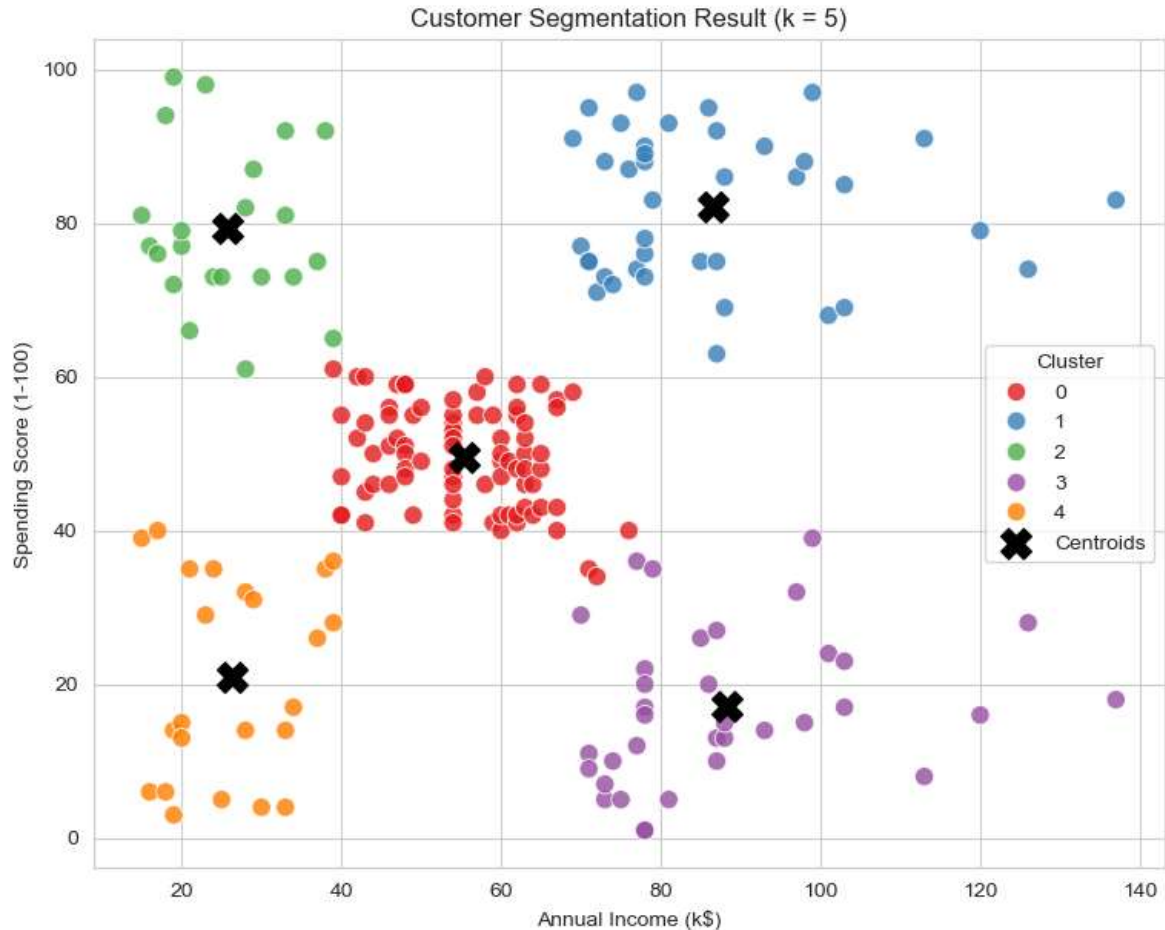
         # Plot cluster centroids (convert back to original scale)
         centroids = scaler.inverse_transform(kmeans_final.cluster_centers_)
         plt.scatter(
```

```

centroids[:, 0], centroids[:, 1],
c='black', s=200, marker='X', label='Centroids'
)

plt.title(f'Customer Segmentation Result (k = {optimal_k})')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend(title='Cluster')
plt.show()

```



```

In [16]: # Calculate average metrics per cluster
cluster_profile = df.groupby('Cluster').agg({
    'Age': 'mean',
    'Annual_Income': 'mean',
    'Spending_Score': 'mean',
    'Gender': 'mean'
}).round(2)

# Rename for readability
cluster_profile.columns = [
    'Avg Age',
    'Avg Annual Income (k$)',
    'Avg Spending Score',
    'Female Ratio'
]

print("=== Cluster Profile Summary ===")
display(cluster_profile)

```

```

=== Cluster Profile Summary ===

```

	Avg Age	Avg Annual Income (k\$)	Avg Spending Score	Female Ratio
Cluster				
0	42.72	55.30	49.52	0.59
1	32.69	86.54	82.13	0.54
2	25.27	25.73	79.36	0.59
3	41.11	88.20	17.11	0.46
4	45.22	26.30	20.91	0.61

Detailed Customer Segmentation Profile

Cluster ID	Customer Type	Core Characteristics	Typical Tags
0	VIP Core Customers	High income & high spending	High net worth, high loyalty
1	High-Income Under-Engaged	High income & low spending	Largest ROI growth potential
2	Impulse Spenders	Young & high willingness to spend	Trend-sensitive, emotional
3	Mass-Market Base	Low income & low spending	Price-sensitive, traffic-driven
4	Stable Mainstream	Balanced all metrics	Rational, cross-sell potential

```
In [17]: # Assumptions: 4 purchases per year, 5-year retention period
repurchase_freq = 4
retention_years = 5

# Calculate simplified CLV
df['Estimated_CLV'] = df['Spending_Score'] * repurchase_freq * retention_years

# Summarize CLV by cluster
clv_summary = df.groupby('Cluster')['Estimated_CLV'].agg(['mean', 'sum']).round(2)
clv_summary.columns = ['Average CLV', 'Total CLV Contribution']

print("=== CLV Estimation by Cluster ===")
display(clv_summary)

=== CLV Estimation by Cluster ===
```

	Average CLV	Total CLV Contribution
Cluster		
0	990.37	80220
1	1642.56	64060
2	1587.27	34920
3	342.29	11980
4	418.26	9620

Targeted Marketing Strategies

1. VIP Core Customers (Cluster 0)

- Focus: Retention & exclusivity
- Actions: Private membership benefits, premium product previews, birthday rewards, personal shopping service

2. High-Income Under-Engaged Customers (Cluster 1)

- Focus: Activation & penetration
- Actions: Precision marketing for high-end categories, exclusive member-day discounts, loyalty program upgrade offers

3. Impulse Spenders (Cluster 2)

- Focus: Frequency & new product conversion
- Actions: Flash sales, trending product pushes, new arrival notifications, limited-edition collections

4. Mass-Market Base Customers (Cluster 3)

- Focus: Foot traffic & upsell
- Actions: Value bundles, point redemption campaigns, seasonal discount events

5. Stable Mainstream Customers (Cluster 4)

- Focus: Cross-selling & basket size
- Actions: Complementary product recommendations, bundle deals, family-oriented promotions

Core Conclusions

1. Mall customers naturally form 5 distinct value segments. Homogeneous marketing leads to significant resource misallocation.
2. High-income low-consumption customers represent the highest ROI growth opportunity. Activating this group can drive overall revenue uplift.
3. Younger customers show stronger consumption willingness, which supports targeted positioning of youth-oriented categories and marketing campaigns.

```

In [2]: # =====
# Customer Segmentation Full Pipeline (One-click Run)
# Includes: Data Generation/Loading → Clustering → Effect Validation → Business
# =====

# ===== 0. Import All Dependencies =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# Formatting settings
plt.rcParams['axes.unicode_minus'] = False

# =====
# Part 1: Generate Simulated Data (Comment this out if you have real data)
# =====
np.random.seed(42)
n_samples = 500

# Build 5 typical customer segments
cluster1 = pd.DataFrame({
    'CustomerID': [f'C{str(i).zfill(4)}' for i in range(1, 101)],
    'Annual_Income': np.random.normal(80000, 12000, 100),
    'Total_Spend': np.random.normal(35000, 5000, 100),
    'Purchase_Frequency': np.random.normal(24, 4, 100),
    'Average_Order_Value': np.random.normal(1450, 200, 100),
    'Age': np.random.normal(38, 6, 100)
})
cluster2 = pd.DataFrame({
    'CustomerID': [f'C{str(i).zfill(4)}' for i in range(101, 201)],
    'Annual_Income': np.random.normal(75000, 10000, 100),
    'Total_Spend': np.random.normal(12000, 2500, 100),
    'Purchase_Frequency': np.random.normal(8, 2, 100),
    'Average_Order_Value': np.random.normal(1500, 250, 100),
    'Age': np.random.normal(42, 7, 100)
})
cluster3 = pd.DataFrame({
    'CustomerID': [f'C{str(i).zfill(4)}' for i in range(201, 351)],
    'Annual_Income': np.random.normal(45000, 8000, 150),
    'Total_Spend': np.random.normal(18000, 3000, 150),
    'Purchase_Frequency': np.random.normal(15, 3, 150),
    'Average_Order_Value': np.random.normal(1200, 180, 150),
    'Age': np.random.normal(32, 5, 150)
})
cluster4 = pd.DataFrame({
    'CustomerID': [f'C{str(i).zfill(4)}' for i in range(351, 431)],
    'Annual_Income': np.random.normal(30000, 5000, 80),
    'Total_Spend': np.random.normal(6000, 1500, 80),
    'Purchase_Frequency': np.random.normal(6, 2, 80),
    'Average_Order_Value': np.random.normal(1000, 150, 80),
    'Age': np.random.normal(25, 3, 80)
})
cluster5 = pd.DataFrame({
    'CustomerID': [f'C{str(i).zfill(4)}' for i in range(431, 501)],
    'Annual_Income': np.random.normal(55000, 9000, 70),
    'Total_Spend': np.random.normal(22000, 4000, 70),

```

```

    'Purchase_Frequency': np.random.normal(5, 1.5, 70),
    'Average_Order_Value': np.random.normal(4400, 600, 70),
    'Age': np.random.normal(45, 8, 70)
})

# Combine, shuffle and export
df = pd.concat([cluster1, cluster2, cluster3, cluster4, cluster5], ignore_index=True)
df = df.sample(frac=1, random_state=42).reset_index(drop=True)
for col in ['Annual_Income', 'Total_Spend', 'Purchase_Frequency', 'Average_Order_Value']:
    df[col] = df[col].round(0).astype(int)

df.to_csv('customer_data.csv', index=False, encoding='utf-8-sig')
print("✅ Simulated data generated: customer_data.csv")

# =====
# Part 2: Data Loading & Clustering Model Training
# =====
print("\n" + "="*50)
print("📊 Part 2: Clustering Training & Effect Validation")
print("="*50)

# Load data
df = pd.read_csv("customer_data.csv", encoding='utf-8-sig')

# Select numeric features for clustering
feature_cols = ['Annual_Income', 'Total_Spend', 'Purchase_Frequency', 'Average_Order_Value']
features = df[feature_cols].dropna()

# Standardize features
scaler = StandardScaler()
X = scaler.fit_transform(features)

# ----- 2.1 Elbow Method for Optimal k -----
sse = []
k_range = range(2, 11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X)
    sse.append(kmeans.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(k_range, sse, 'o-', color='#2c7be5')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Within-Cluster Sum of Squares (SSE)')
plt.title('Elbow Method for Optimal k Value')
plt.grid(alpha=0.3)
plt.show()

# ----- 2.2 Silhouette Score Validation (k=5) -----
kmeans_final = KMeans(n_clusters=5, random_state=42, n_init=10)
labels = kmeans_final.fit_predict(X)
avg_score = silhouette_score(X, labels)
print(f"✅ Average silhouette score for k=5: {avg_score:.3f} (0.4-0.6 indicates good clustering)")

# ----- 2.3 Cluster Size Distribution -----
df = df.dropna(subset=feature_cols).copy()
df['Cluster'] = labels
cluster_dist = df['Cluster'].value_counts(normalize=True).sort_index().round(4)
print("\n📊 Cluster size distribution:")
for c, p in cluster_dist.items():

```

```

    print(f"    Cluster {c}: {p:.1f}%")

# =====
# Part 3: Business Logic Validation
# =====

print("\n" + "="*50)
print("💡 Part 3: Business Logic Validation")
print("="*50)

# ----- 3.1 Generate Standard Segment Profile Table -----
cluster_profile = df.groupby('Cluster').agg({
    'Annual_Income': ['mean', 'median'],
    'Total_Spend': ['mean', 'median'],
    'Purchase_Frequency': ['mean', 'median'],
    'Average_Order_Value': ['mean', 'median'],
    'Age': ['mean', 'median'],
    'CustomerID': 'count'
}).round(0)
cluster_profile.columns = [
    'Avg_Annual_Income', 'Median_Annual_Income',
    'Avg_Total_Spend', 'Median_Total_Spend',
    'Avg_Purchase_Frequency', 'Median_Purchase_Frequency',
    'Avg_AOV', 'Median_AOV',
    'Avg_Age', 'Median_Age',
    'Count'
]
cluster_profile['Share'] = (cluster_profile['Count'] / len(df) * 100).round(1).a

print("\n📄 Customer segment profile:")
print(cluster_profile)

# ----- 3.2 VIP Segment Dual High Validation -----
income_top20 = df['Annual_Income'].quantile(0.8)
spend_top20 = df['Total_Spend'].quantile(0.8)
vip_cluster = cluster_profile['Avg_Total_Spend'].idxmax()
vip_income = cluster_profile.loc[vip_cluster, 'Avg_Annual_Income']
vip_spend = cluster_profile.loc[vip_cluster, 'Avg_Total_Spend']

print(f"\n🔍 VIP segment validation (high income + high spend):")
print(f"    Top 20% income threshold: {income_top20:.0f}")
print(f"    Top 20% spend threshold: {spend_top20:.0f}")
print(f"    VIP cluster (Cluster {vip_cluster}) income meets threshold: {vip_inco")
print(f"    VIP cluster (Cluster {vip_cluster}) spend meets threshold: {vip_spend")

# ----- 3.3 Young Segment Activity Validation -----
youngest_cluster = cluster_profile['Avg_Age'].idxmin()
oldest_cluster = cluster_profile['Avg_Age'].idxmax()
young_freq = cluster_profile.loc[youngest_cluster, 'Avg_Purchase_Frequency']
old_freq = cluster_profile.loc[oldest_cluster, 'Avg_Purchase_Frequency']

print(f"\n🔍 Young segment activity validation:")
print(f"    Youngest cluster (Cluster {youngest_cluster}): {cluster_profile.loc[y")
print(f"    Oldest cluster (Cluster {oldest_cluster}): {cluster_profile.loc[oldest")
print(f"    Frequency ratio (young/old): {young_freq/old_freq:.2f}    (≥1.5 aligns")

# ----- 3.4 High-Income Low-Spend Segment Size Validation -----
income_median = df['Annual_Income'].median()
spend_median = df['Total_Spend'].median()
potential_clusters = cluster_profile[
    (cluster_profile['Avg_Annual_Income'] >= income_median) &

```

```

    (cluster_profile['Avg_Total_Spend'] < spend_median)
]
total_pct = potential_clusters['Count'].sum() / len(df) * 100

print(f"\n🔍 High-income low-spend segment size validation:")
print(f"    Target clusters: {list(potential_clusters.index)}")
print(f"    Total share: {total_pct:.1f}% (15%-30% is reasonable range)")

# ----- 3.5 Quantile Cross-Validation -----
# Convert qcut output to string to avoid Categorical concatenation error
df['Income_Tier'] = pd.qcut(df['Annual_Income'], q=[0, 0.2, 0.8, 1], labels=['Lo
df['Spend_Tier'] = pd.qcut(df['Total_Spend'], q=[0, 0.2, 0.8, 1], labels=['Low_S

# String concatenation now works normally
df['Manual_Segment'] = df['Income_Tier'] + '-' + df['Spend_Tier']

cross_tab = pd.crosstab(df['Cluster'], df['Manual_Segment'])
max_per_cluster = cross_tab.max(axis=1)
overall_match = max_per_cluster.sum() / len(df) * 100

print(f"\n🔍 Clustering vs. manual segmentation match rate: {overall_match:.1f}%

# =====
# Part 4: CLV Calculation & Result Validation
# =====
print("\n" + "="*50)
print("💰 Part 4: CLV Calculation & Result Validation")
print("="*50)

# ----- 4.1 CLV Calculation (Basic: AOV × Frequency × 3-Year Lifespan) -----
df['CLV'] = df['Average_Order_Value'] * df['Purchase_Frequency'] * 3
cluster_clv = df.groupby('Cluster')['CLV'].agg(['mean', 'median']).round(0)
cluster_clv.columns = ['Avg_CLV', 'Median_CLV']

print("\n📈 Customer Lifetime Value (CLV) by segment:")
print(cluster_clv)

# ----- 4.2 Sample for Manual Verification -----
sample = df.sample(5, random_state=42)[['CustomerID', 'Average_Order_Value', 'Pu
print("\n📋 Sample records for manual verification (copy to Excel to cross-check
print(sample)

# ----- 4.3 Full Outlier Check -----
clv_mean = df['CLV'].mean()
negative_count = len(df[df['CLV'] <= 0])
null_count = df['CLV'].isnull().sum()
extreme_count = len(df[df['CLV'] > clv_mean * 5])

print(f"\n⚠️ Outlier check:")
print(f"    Overall CLV mean: {clv_mean:.0f}")
print(f"    Negative CLV records: {negative_count}")
print(f"    Null CLV records: {null_count}")
print(f"    Extreme values (>5× mean): {extreme_count} records, {extreme_count/le

# =====
# Final Result Export
# =====
df.to_csv('customer_segmentation_with_CLV.csv', index=False, encoding='utf-8-sig
cluster_profile.to_csv('segment_profile_summary.csv', encoding='utf-8-sig')

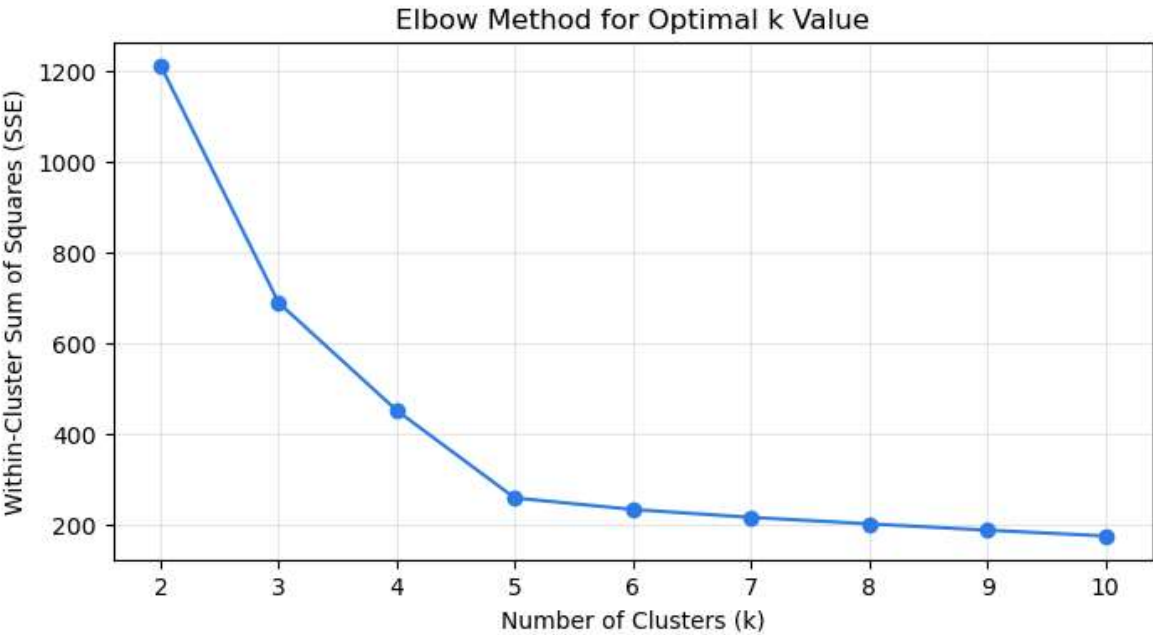
```

```
print("\n" + "="*50)
print("🎉 Full pipeline completed! Exported files:")
print("  1. customer_data.csv — Raw customer data")
print("  2. customer_segmentation_with_CLV.csv — Detailed data with cluster lab")
print("  3. segment_profile_summary.csv — Core metrics summary by segment")
print("="*50)
```

✔ Simulated data generated: customer_data.csv

Part 2: Clustering Training & Effect Validation

```
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
```



✓ Average silhouette score for k=5: 0.577 (0.4–0.6 indicates good segmentation)

✦ Cluster size distribution:

Cluster 0: 20.0%
 Cluster 1: 16.0%
 Cluster 2: 14.0%
 Cluster 3: 30.2%
 Cluster 4: 19.8%

💡 Part 3: Business Logic Validation

📄 Customer segment profile:

	Avg_Annual_Income	Median_Annual_Income	Avg_Total_Spend \
Cluster			
0	78754.0	78476.0	35112.0
1	29338.0	29426.0	6007.0
2	55502.0	54700.0	21669.0
3	46544.0	46796.0	18012.0
4	74085.0	74339.0	12067.0

	Median_Total_Spend	Avg_Purchase_Frequency \
Cluster		
0	35420.0	24.0
1	5955.0	6.0
2	21813.0	5.0
3	18122.0	15.0
4	11918.0	8.0

	Median_Purchase_Frequency	Avg_AOV	Median_AOV	Avg_Age	Median_Age \
Cluster					
0	24.0	1471.0	1460.0	38.0	38.0
1	6.0	988.0	996.0	25.0	25.0
2	6.0	4328.0	4328.0	46.0	45.0
3	15.0	1208.0	1203.0	33.0	33.0
4	8.0	1557.0	1537.0	42.0	42.0

	Count	Share
Cluster		
0	100	20.0%
1	80	16.0%
2	70	14.0%
3	151	30.2%
4	99	19.8%

🔍 VIP segment validation (high income + high spend):

Top 20% income threshold: 76392

Top 20% spend threshold: 27680

VIP cluster (Cluster 0) income meets threshold: True

VIP cluster (Cluster 0) spend meets threshold: True

🔍 Young segment activity validation:

Youngest cluster (Cluster 1): 25 years old, 6.0 purchases/year

Oldest cluster (Cluster 2): 46 years old, 5.0 purchases/year

Frequency ratio (young/old): 1.20 (≥1.5 aligns with business common sense)

🔍 High-income low-spend segment size validation:

Target clusters: [4]

Total share: 19.8% (15%–30% is reasonable range)

🔍 Clustering vs. manual segmentation match rate: 75.6% (≥70% is considered reliable)

=====
💰 Part 4: CLV Calculation & Result Validation
=====

📌 Customer Lifetime Value (CLV) by segment:

	Avg_CLV	Median_CLV
Cluster		
0	107014.0	104018.0
1	18271.0	17775.0
2	69171.0	68340.0
3	54522.0	53760.0
4	37058.0	37344.0

📋 Sample records for manual verification (copy to Excel to cross-check):

	CustomerID	Average_Order_Value	Purchase_Frequency	CLV
361	C0293	1499	21	94437
73	C0291	1195	16	57360
374	C0289	989	15	44505
155	C0358	868	5	13020
104	C0333	1017	18	54918

⚠️ Outlier check:

Overall CLV mean: 57813
Negative CLV records: 1
Null CLV records: 0
Extreme values (>5× mean): 0 records, 0.0%

=====
🎉 Full pipeline completed! Exported files:
1. customer_data.csv — Raw customer data
2. customer_segmentation_with_CLV.csv — Detailed data with cluster labels and CLV
3. segment_profile_summary.csv — Core metrics summary by segment
=====

D:\Program Files\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1419: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=2.
warnings.warn(